

Comparaison de RSA et ElGamal

dans le cadre du chiffrement léger

Basile Capier

n° 45879

TIPE 2025–2026

Introduction

Cadre théorique

Protocole et résultats Python

Étude sur JavaCard

Conclusion

Introduction

Contexte : cryptographie sous contrainte

Les algorithmes cryptographiques classiques sont conçus pour des machines à *ressources abondantes*.

Mais une part croissante de nos systèmes fonctionne avec :

- Quelques Ko de RAM
- CPU à quelques MHz
- Alimentation par batterie

⇒ **Chiffrement léger** : adaptation des algorithmes à ces contraintes.

Cibles typiques

- Cartes SIM
- Cartes bancaires EMV
- Passeports biométriques
- Objets connectés (IoT)
- Capteurs médicaux

Deux algorithmes, deux structures

RSA (1977)

- Arithmétique modulaire dans $\mathbb{Z}/n\mathbb{Z}$, $n = pq$
- Sécurité $\equiv$ **factoriser** n
- Très utilisé : SSL/TLS, VPN, signatures

ElGamal (1985)

- Groupe cyclique $(\mathbb{Z}/p\mathbb{Z})^*$
- Sécurité $\equiv$ **logarithme discret**
- Base de nombreuses variantes (DSA, ECC, ECIES)

Problématique

Quel impact peut avoir le caractère cyclique d'un groupe sur les performances et la sécurité des algorithmes de cryptage léger ?

Cadre théorique

Génération des clés

1. p, q premiers ; $n = pq$
2. $\varphi(n) = (p - 1)(q - 1)$
3. e tel que $\gcd(e, \varphi(n)) = 1$ (typ. $e = 65\,537 = 2^{16} + 1$)
4. $d \equiv e^{-1} \pmod{\varphi(n)}$

Clé publique : (n, e)

Clé privée : (n, d)

Chiffrement

$$c \equiv m^e \pmod{n}$$

Déchiffrement

$$m \equiv c^d \pmod{n}$$

Théorème d'Euler. Si $\gcd(m, n) = 1$, alors $m^{\varphi(n)} \equiv 1 \pmod{n}$.

Correction de RSA. On a $ed \equiv 1 \pmod{\varphi(n)}$, donc il existe k tel que $ed = 1 + k\varphi(n)$. D'où

$$c^d = (m^e)^d = m^{ed} = m^{1+k\varphi(n)} = m \cdot (m^{\varphi(n)})^k \equiv m \pmod{n}.$$

Génération des clés

1. p premier, g générateur de $(\mathbb{Z}/p\mathbb{Z})^*$
2. x aléatoire dans $\{1, \dots, p-2\}$
3. $y = g^x \bmod p$

Clé publique : (p, g, y)

Clé privée : x

Chiffrement (aléa k)

$$c_1 = g^k, \quad c_2 = m \cdot y^k$$

Déchiffrement

$$m = c_2 \cdot (c_1^x)^{-1} \pmod{p}$$

ElGamal — exploiter la structure cyclique

Exploiter la cyclicité : sous-groupe d'ordre premier $q \mid (p - 1)$, avec $|q| \ll |p|$.

Paramètres :

1. Tirer q premier de $|q|$ bits (typ. 256).
2. Tirer h aléatoire jusqu'à ce que $p = hq + 1$ soit premier.
3. Construire $g = \alpha^h \bmod p$ pour α aléatoire, jusqu'à $g \neq 1$.

Génération des clés : $x \in \{1, \dots, q - 1\}$ aléatoire (*exposant court*), $y = g^x \bmod p$.

Chiffrement (*aléa k court*)

$$c_1 = g^k, \quad c_2 = m \cdot y^k$$

Déchiffrement

$$m = c_2 \cdot (c_1^x)^{-1} \pmod{p}$$

Correction.

$$c_2 \cdot (c_1^x)^{-1} = (m \cdot y^k) \cdot (g^{kx})^{-1} = m \cdot g^{xk} \cdot g^{-xk} = m \pmod{p}.$$

Pourquoi g est d'ordre q ? Par Fermat, $\alpha^{p-1} \equiv 1$; comme $p-1 = hq$,

$$g^q = (\alpha^h)^q = \alpha^{p-1} \equiv 1 \pmod{p}.$$

Donc $\text{ord}(g) \mid q$; q premier $\Rightarrow \text{ord}(g) \in \{1, q\}$; et $g \neq 1 \Rightarrow \text{ord}(g) = q$.

Conséquence sur les performances

x, k dans $\{1, \dots, q-1\}$: $|q|$ **bits seulement**, au lieu de $|p|$ bits dans le groupe entier.

Protocole et résultats Python

Protocole expérimental

Implémentations

Implémentation *volontairement symétrique* en Python :

- Mêmes primitives (getPrime, pow, random)
- Même algorithme d'exponentiation (square-and-multiply de GMP)
- Neutralise les biais d'implémentation

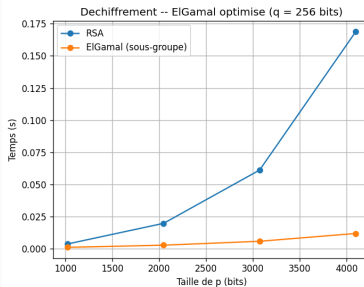
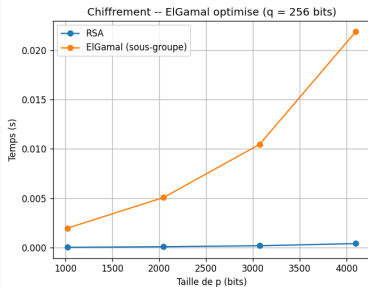
Mesures

Pour chaque taille de $p \in \{1024, 2048, 3072, 4096\}$ bits, avec $|q| = 256$ bits pour ElGamal : génération des clés, chiffrement, déchiffrement.

Simulation de l'environnement contraint via Docker

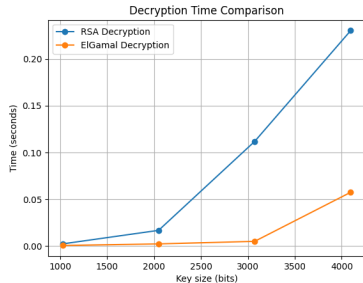
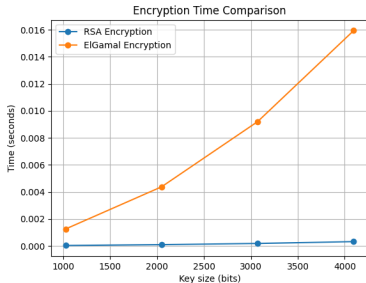
```
docker run --memory="256m" --cpus="0.5" crypto-test
```

Résultats — pleine puissance



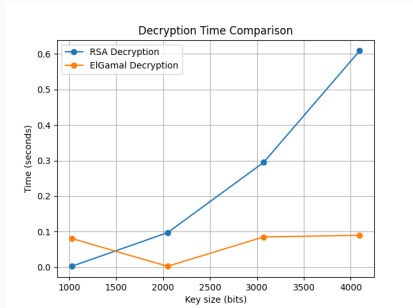
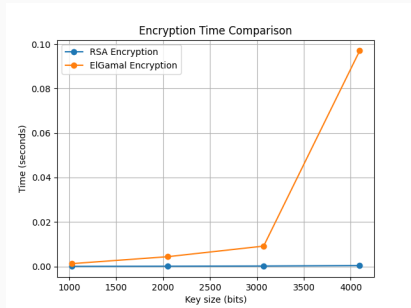
- **Chiffrement** : RSA imbattable ($e = 65\,537$ très court). ElGamal sous 25 ms même à 4096 bits.
- **Déchiffrement à 4096 bits** : RSA 169 ms vs ElGamal 12 ms ($\sim 14\times$ plus rapide).

Résultats — 256 Mo / 50 % CPU



- Déchiffrement à 4096 bits : RSA ~230 ms, ElGamal ~57 ms ⇒
~4× plus rapide.
- L'avantage d'ElGamal persiste sous contrainte.

Résultats — 128 Mo / 20 % CPU



- Déchiffrement à 4096 bits : RSA ~610 ms, ElGamal ~90 ms $\Rightarrow$ **$\sim 7\times$ plus rapide.**
- Bruit notable à 1024 bits (mesure unique, non moyennée).

Configuration	RSA déchiffr. 4096	ElGamal déchiffr. 4096	Ratio
Pleine puissance	169ms	12ms	14×
256 Mo / 50 % CPU	230ms	57ms	4×
128 Mo / 20 % CPU	610ms	90ms	7×

Conclusion Python

Dans toutes les configurations testées, ElGamal est plus rapide que RSA en déchiffrement. L'exploitation explicite de la structure cyclique donne un avantage robuste, indépendant des contraintes Docker.

Critique du protocole Python

- Python est un langage interprété, très éloigné du matériel : la mesure capte du bruit système (garbage collector, gestion mémoire).
- Docker limite les ressources, mais ne simule pas un *vrai* environnement embarqué (mêmes instructions, même OS).
- Mesures non moyennées (1 exécution par taille) : tendances qualitatives fiables, valeurs précises bruyantes.

Transition

Besoin d'un **environnement véritablement contraint** et représentatif du déploiement réel : la JavaCard.

Étude sur JavaCard

Pourquoi la JavaCard ?

Une **JavaCard** est une carte à puce embarquant une JVM réduite.

Caractéristiques typiques

- Quelques Ko de RAM, mémoire EEPROM persistante
- Processeur 16/32 bits à quelques MHz
- API spartiate (pas de BigInteger, allocations statiques)

Atout majeur pour notre étude : ces cartes embarquent un **coprocesseur cryptographique matériel**, généralement dédié à RSA.

⇒ La comparaison avec ElGamal devient cruciale : a-t-il accès au même matériel ?

RSA (BenchmarkApplet)

- Utilise directement l'API `Cipher.ALG_RSA_PKCS1`
- Exécuté par le **coprocesseur matériel**

ElGamal (ElGamalApplet)

- Aucun équivalent natif dans l'API JavaCard
- Implémentation manuelle avec détournement du moteur RSA pour utiliser le coprocesseur sur les exponentiations
- Surcoût logiciel important (gestion de c_1 , c_2 , inverse modulaire)

- **Taille des buffers** : le chiffré ElGamal (c_1, c_2) fait 256 octets pour une clé de 1024 bits.
⇒ Logique de récupération en deux temps (getC1, getC2).
- **Pas d'inverse modulaire natif**. Solution mathématique : par le petit théorème de Fermat,

$$a^{-1} \equiv a^{p-2} \pmod{p}$$

Une inversion devient une exponentiation ⇒ exploitable par le coprocesseur.

Opération	Temps (ms)	Ratio RSA
RSA chiffrement	2,22	1×
RSA déchiffrement	4,56	1×
ElGamal chiffrement	32,2	~14×
ElGamal déchiffrement	293	~64×

Renversement par rapport à Python

Sur JavaCard, RSA bénéficie de l'**accélération matérielle complète** du coprocesseur, alors qu'ElGamal n'en bénéficie que partiellement. L'écart algorithmique en faveur d'ElGamal est plus que compensé.

Conclusion

Réponse à la problématique

Deux expériences, deux verdicts apparemment opposés :

- **Python** : ElGamal exploitant la cyclicité gagne ($4\text{-}14\times$ plus rapide que RSA en déchiffrement).
- **JavaCard** : RSA gagne largement ($\sim 64\times$ en déchiffrement).

Synthèse

La structure algébrique *influence* les performances — mais le sens de l'avantage dépend de ce que l'on optimise :

- **Avantage algorithmique** (Python) : la cyclicité permet des exposants courts, ElGamal gagne.
- **Avantage matériel** (JavaCard) : RSA a trente ans d'optimisation hardware dédiée.

Les deux structures ont chacune leur domaine d'application,

Cryptographie sur courbes elliptiques (ECC)

ElGamal transposé à un autre groupe cyclique : les points d'une courbe elliptique sur $\mathbb{F}_p$.

- Logarithme discret beaucoup plus difficile
- 256 bits ECC $\approx$ 3072 bits RSA en sécurité
- Coprocesseurs ECC déjà répandus sur les JavaCards modernes

Menace quantique

RSA et ElGamal cassés par l'algorithme de Shor sur ordinateur quantique.

⇒ Cryptographie post-quantique

Annexe — Code Python : imports

```
import time
import random
import secrets
import matplotlib.pyplot as plt

from Crypto.Util.number import getPrime, isPrime, inverse, bytes_to_long,
    long_to_bytes
```

Annexe — Code Python : RSA

```
def my_rsa_generate_keys(key_size):
    p, q = getPrime(key_size // 2), getPrime(key_size // 2)
    n = p * q
    phi_n = (p - 1) * (q - 1)
    e = 65537
    d = pow(e, -1, phi_n)
    return (n, e), (n, d)

def my_rsa_encrypt(message, public_key):
    return pow(message, public_key[1], public_key[0])

def my_rsa_decrypt(cipher, private_key):
    return pow(cipher, private_key[1], private_key[0])

def my_rsa_test(secret_message, public_key, private_key):
    message = bytes_to_long(secret_message)

    start = time.perf_counter()
    cipher = my_rsa_encrypt(message, public_key)
    enc_time = time.perf_counter() - start

    start = time.perf_counter()
    decipher = my_rsa_decrypt(cipher, private_key)
    dec_time = time.perf_counter() - start

    assert decipher == message, "RSA : echec de dechiffrement"
    return enc_time, dec_time
```

Annexe — ElGamal : génération des paramètres

```
def generate_dsa_like_params(p_size, q_size):
    q = getPrime(q_size)
    h_size = p_size - q_size
    while True:
        h = secrets.randbits(h_size - 1) | (1 << (h_size - 1))
        if h % 2 == 1:
            h += 1
        p = h * q + 1
        if p.bit_length() == p_size and isPrime(p):
            break
    while True:
        alpha = random.randint(2, p - 2)
        g = pow(alpha, h, p)
        if g != 1:
            break
    return p, q, g

def elgamal_generate_keys(p, q, g):
    x = random.randint(1, q - 1)
    y = pow(g, x, p)
    return x, y
```

Annexe — ElGamal : chiffrement/déchiffrement

```
def elgamal_encrypt(p, q, g, y, message):
    k = random.randint(1, q - 1)
    c1 = pow(g, k, p)
    c2 = (message * pow(y, k, p)) % p
    return c1, c2

def elgamal_decrypt(p, x, c1, c2):
    s = pow(c1, x, p)
    s_inv = inverse(s, p)
    return (c2 * s_inv) % p

def elgamal_test(secret_message, params, x, y):
    p, q, g = params
    message = bytes_to_long(secret_message)

    start = time.perf_counter()
    c1, c2 = elgamal_encrypt(p, q, g, y, message)
    enc_time = time.perf_counter() - start

    start = time.perf_counter()
    decrypted = elgamal_decrypt(p, x, c1, c2)
    dec_time = time.perf_counter() - start

    assert decrypted == message, "ElGamal : echec de dechiffrement"
    return enc_time, dec_time
```

Annexe — Benchmark

```
key_sizes = [1024, 2048, 3072, 4096]

rsa_enc_times, rsa_dec_times, elg_enc_times, elg_dec_times, rsa_key_times,
    elg_key_times = [], [], [], [], [], []

secret_message = long_to_bytes(secrets.randbits(256))

for size in key_sizes:
    print(f"Testing key size: {size} bits")

    L_r_enc, L_r_dec, L_e_enc, L_e_dec, L_r_kt, L_e_kt = [], [], [], [], [], []

    start = time.perf_counter()
    p, q, g = generate_dsa_like_params(size, 256)
    x, y = elgamal_generate_keys(p, q, g)
    elgamal_param = (p, q, g), x, y
    elg_key_times.append(time.perf_counter()-start)

    start = time.perf_counter()
    rsa_param = my_rsa_generate_keys(size)
    rsa_key_times.append(time.perf_counter()-start)

    r_enc, r_dec = my_rsa_test(secret_message, rsa_param[0], rsa_param[1])
    e_enc, e_dec = elgamal_test(secret_message, elgamal_param[0], elgamal_param
        [1], elgamal_param[2])

    rsa_enc_times.append(r_enc)
    rsa_dec_times.append(r_dec)
    elg_enc_times.append(e_enc)
    elg_dec_times.append(e_dec)
```

```
# Encryption graph
plt.figure()
plt.plot(key_sizes, rsa_enc_times, marker='o', label='RSA Encryption')
plt.plot(key_sizes, elg_enc_times, marker='o', label='ElGamal Encryption')
plt.xlabel("Key size (bits)")
plt.ylabel("Time (seconds)")
plt.title("Encryption Time Comparison (Key Generation Excluded)")
plt.legend()
plt.grid()
plt.show()
```


Annexe — BenchmarkApplet

```
package fr.tipe;

import javacard.framework.*;
import javacard.security.*;
import javacardx.apdu.ExtendedLength;
import javacardx.crypto.Cipher;

public class BenchmarkApplet extends Applet implements ExtendedLength {

    private static final byte[] ELGAMAL_APPLET_AID = {
        (byte)0xFF, (byte)0x53, (byte)0x56, (byte)0x6F, (byte)0x78, (byte)0
            xFF, (byte)0x22, (byte)0x12, (byte)0x02
    };

    private static final byte CRYPTO_CLA = (byte) 0x22;
    private static final byte RSA_INS = (byte) 0x10;
    private static final byte ELGAMAL_INS = (byte) 0x20;

    private ElGamalInterface elGamalApp;

    private final RSAPublicKey rsaPublicKey;
    private final RSAPrivateKey rsaPrivateKey;
    private final Cipher rsaCipher;

    private final static byte[] HELLO_WORLD_MESSAGE = { (byte) 0x62, (byte) 0x6F
        , (byte) 0x6E, (byte) 0x6A, (byte) 0x6F, (byte) 0x75, (byte) 0x72 };
    private final static byte HELLO_WORLD = (byte) 0xFF;
```

Annexe — BenchmarkApplet

```
private final byte[] resultArray_128;
private final byte[] elgamalResult;
private final byte[] elgamalOutput;

private boolean isBufferAllZeros(byte[] buffer) {
    for (short i = 0; i < (short)buffer.length; i++) {
        if (buffer[i] != (byte)0x00) {
            return false;
        }
    }
    return true;
}

private void encryptRSA(byte[] inBuff, short InOffset, short inLength, byte
[] outBuff) {
    rsaCipher.init(rsaPublicKey, Cipher.MODE_ENCRYPT);
    rsaCipher.doFinal(inBuff, InOffset, inLength, outBuff, (short) 0);
}

private void decryptRSA(byte[] inBuff, short InOffset, short inLength, byte
[] outBuff) {
    rsaCipher.init(rsaPrivateKey, Cipher.MODE_DECRYPT);
    rsaCipher.doFinal(inBuff, InOffset, inLength, outBuff, (short) 0);
}

private void returnData (APDU apdu, byte[] buffer, byte[] retdata, short
retOff, short length) {
    Util.arrayCopy(retdata, retOff, buffer, ISO7816.OFFSET_CDATA, length);
    apdu.setOutgoing();
    apdu.setOutgoingLength(length);
    apdu.sendBytes(ISO7816.OFFSET_CDATA, length);
}
```

Annexe — BenchmarkApplet

```
private void returnLongData (APDU apdu, byte[] buffer, byte[] retdata, short
    length) {
    Util.arrayCopy(retdata, (short) 0, buffer, ISO7816.OFFSET_CDATA, length)
        ;
    apdu.setOutgoing();
    apdu.setOutgoingLength(length);
    apdu.sendBytesLong(buffer, ISO7816.OFFSET_CDATA, length);
}

public static void install (byte[] bArray, short bOffset, byte bLength) {
    new BenchmarkApplet(); }

protected BenchmarkApplet() {
    register();

    resultArray_128 = JCSysyem.makeTransientByteArray((short)128, JCSysyem.
        CLEAR_ON_RESET);
    elgamalResult = new byte[(short)256];
    elgamalOutput = new byte[(short)128];
    try {
        AID elGamalAppletAID = JCSysyem.lookupAID(ELGAMAL_APPLET_AID, (short
            ) 0, (byte) ELGAMAL_APPLET_AID.length);
        if (elGamalAppletAID != null) {
            this.elGamalApp = (ElGamalInterface) JCSysyem.
                getAppletShareableInterfaceObject(elGamalAppletAID, (byte)
                    0x01);
        }
        if (this.elGamalApp == null) {
            ISOException.throwIt(ISO7816.SW_FILE_NOT_FOUND);
        }
    }
```

```
rsaPublicKey = (RSAPublicKey) RSA_KeyPair.getPublic();
rsaPrivateKey = (RSAPrivateKey) RSA_KeyPair.getPrivate();
rsaCipher = Cipher.getInstance(Cipher.ALG_RSA_PKCS1, false);
}

@Override
public void process (APDU apdu) throws ISOException {
    if (selectingApplet()) return;

    byte[] buffer = apdu.getBuffer();

    if (buffer[ISO7816.OFFSET_CLA] != CRYPTO_CLA) {
        ISOException.throwIt(ISO7816.SW_CLA_NOT_SUPPORTED);
    }
}
```

```
switch (buffer[ISO7816.OFFSET_INS]) {
    case RSA_INS:
        if (buffer[ISO7816.OFFSET_LC] == (byte)0) ISOException.throwIt(
            ISO7816.SW_DATA_INVALID);
        else {
            short totalBytesRead = apdu.setIncomingAndReceive();
            short dataOffset = apdu.getOffsetCdata();
            if (buffer[ISO7816.OFFSET_P1] == (byte) 0x01) {
                encryptRSA(buffer, dataOffset, totalBytesRead,
                    resultArray_128);
                returnData(apdu, buffer, resultArray_128, (short)0, (
                    short)128);
            } else if (buffer[ISO7816.OFFSET_P1] == (byte) 0x02) {
                if (buffer[ISO7816.OFFSET_LC] == (byte)128) {
                    Util.arrayFill(resultArray_128, (short)0, (short)
                        128, (byte)0x00);
                    decryptRSA(buffer, dataOffset, totalBytesRead,
                        resultArray_128);
                    returnData(apdu, buffer, resultArray_128, (short)0,
                        (short)128);
                } else ISOException.throwIt(ISO7816.SW_DATA_INVALID);
            } else ISOException.throwIt(ISO7816.SW_INCORRECT_P1P2);
        }
    }
    break;
```

Annexe — BenchmarkApplet

```
case ELGAMAL_INS:
    if (buffer[ISO7816.OFFSET_P1] == (byte)0x01) {
        if (buffer[ISO7816.OFFSET_P2] == (byte)0x00) {
            if (buffer[ISO7816.OFFSET_LC] == (byte)0) ISOException.
                throwIt(ISO7816.SW_DATA_INVALID);
            else {
                short totalBytesRead = apdu.setIncomingAndReceive();
                byte[] tempRes = this.elGamalApp.cmdEncrypt(buffer,
                    totalBytesRead);
                Util.arrayCopy(tempRes, (short)0, elgamalResult, (
                    short)0, (short)256);
            }
        } else if (buffer[ISO7816.OFFSET_P2] == (byte)0xC1) {
            if (isBufferAllZeros(elgamalResult)) {
                ISOException.throwIt(ISO7816.SW_FILE_NOT_FOUND);
            } else {
                returnData(apdu, buffer, elgamalResult, (short)0, (
                    short)128);
            }
        } else if (buffer[ISO7816.OFFSET_P2] == (byte)0xC2) {
            if (isBufferAllZeros(elgamalResult)) {
                ISOException.throwIt(ISO7816.SW_FILE_NOT_FOUND);
            } else {
                returnData(apdu, buffer, elgamalResult, (short)128,
                    (short)128);
            }
        } else ISOException.throwIt(ISO7816.SW_INCORRECT_P1P2);
    } else if (buffer[ISO7816.OFFSET_P1] == (byte)0x02) {
        if (buffer[ISO7816.OFFSET_P2] == (byte)0x00) {
            byte[] tempRes = this.elGamalApp.cmdDecrypt(
```

```
    case HELLO_WORLD:
        if (buffer[ISO7816.OFFSET_P1] == (byte) 0x01) returnData(apdu,
            buffer, HELLO_WORLD_MESSAGE, (short)0, (short)7);
        else if (buffer[ISO7816.OFFSET_P1] == (byte) 0x02) returnData(
            apdu, buffer, this.elGamalApp.helloWorld(), (short)0, (
            short)7);
        else ISOException.throwIt(ISO7816.SW_INCORRECT_P1P2);
        break;
    default: {
        ISOException.throwIt(ISO7816.SW_INS_NOT_SUPPORTED);
        break;
    }
}

}

}
```

```
package fr.tipe;

import javacard.framework.Shareable;

public interface ElGamalInterface extends Shareable{
    byte[] cmdEncrypt(byte[] apduBuff, short len);
    byte[] helloWorld();
    byte[] cmdDecrypt(byte[] apduBuff, short len);
}
```



```
package fr.tipe;

import javacard.framework.*;
import javacard.security.*;
import javacardx.crypto.*;

public class ElGamalApplet extends Applet implements ElGamalInterface {

    private boolean isInterfaceShared = false;

    private static final byte[] HELLO_WORLD_MESSAGE_2 = {
        (byte)0x42, (byte)0x6F, (byte)0x6E, (byte)0x6A, (byte)0x6F, (byte)0
        x75, (byte)0x72
    };

    public static final short KEY_LEN = (short)128;
```

Annexe — ElGamalApplet

```
private static final byte[] P_BYTES = {
    (byte)0xf0, (byte)0xbc, (byte)0x29, (byte)0xa8, (byte)0xc7, (byte)0
    xfc, (byte)0x7e, (byte)0xf8,
    (byte)0x96, (byte)0xe3, (byte)0x58, (byte)0xd8, (byte)0xad, (byte)0
    xdb, (byte)0xf8, (byte)0x64,
    (byte)0xb1, (byte)0xeb, (byte)0x7e, (byte)0x82, (byte)0x89, (byte)0
    x12, (byte)0x7d, (byte)0x00,
    (byte)0x39, (byte)0xdb, (byte)0xb5, (byte)0x4e, (byte)0x70, (byte)0
    x01, (byte)0x10, (byte)0x1c,
    (byte)0xdd, (byte)0x93, (byte)0xd3, (byte)0xfb, (byte)0x00, (byte)0
    x89, (byte)0xa1, (byte)0x65,
    (byte)0xb8, (byte)0xcb, (byte)0x34, (byte)0xfd, (byte)0x3f, (byte)0
    xfd, (byte)0x94, (byte)0x84,
    (byte)0xed, (byte)0x63, (byte)0x3b, (byte)0xc7, (byte)0xc0, (byte)0
    xce, (byte)0xfc, (byte)0x77,
    (byte)0xba, (byte)0xc6, (byte)0xbe, (byte)0x3f, (byte)0xb7, (byte)0
    x18, (byte)0x7e, (byte)0x7d,
    (byte)0x3b, (byte)0x63, (byte)0xa5, (byte)0x65, (byte)0xa2, (byte)0
    xb6, (byte)0xae, (byte)0xb5,
    (byte)0x53, (byte)0x8f, (byte)0xe5, (byte)0xb4, (byte)0x43, (byte)0
    x05, (byte)0x33, (byte)0x01,
    (byte)0xeb, (byte)0x0d, (byte)0x90, (byte)0xe7, (byte)0x3b, (byte)0
    xc6, (byte)0xa8, (byte)0x80,
    (byte)0x8e, (byte)0x08, (byte)0x0f, (byte)0x13, (byte)0x68, (byte)0
    xcf, (byte)0x15, (byte)0xb9,
    (byte)0xc5, (byte)0x2e, (byte)0x28, (byte)0x82, (byte)0xeb, (byte)0
    x0a, (byte)0x5a, (byte)0xeb,
    (byte)0xb6, (byte)0x99, (byte)0x6e, (byte)0x4d, (byte)0x18, (byte)0
    xce, (byte)0x3d, (byte)0xef,
    (byte)0xd0, (byte)0xc1, (byte)0x14, (byte)0x2b, (byte)0x60, (byte)0
```

Annexe — ElGamalApplet

```
private static final byte[] G_BYTES = {
    (byte)0x00,
    (byte)0x03, (byte)0x0e, (byte)0x7d, (byte)0xa3, (byte)0xd3, (byte)0
        xbc, (byte)0x64,
    (byte)0xc0, (byte)0x62, (byte)0xb6, (byte)0x24, (byte)0x20, (byte)0
        x81, (byte)0xd5, (byte)0x8b,
    (byte)0xfd, (byte)0xab, (byte)0x26, (byte)0x2c, (byte)0x61, (byte)0
        x90, (byte)0x13, (byte)0x0c,
    (byte)0x2b, (byte)0xe4, (byte)0xce, (byte)0x25, (byte)0x84, (byte)0
        xfe, (byte)0xb5, (byte)0xf6,
    (byte)0xbd, (byte)0x9f, (byte)0x05, (byte)0xa7, (byte)0xcf, (byte)0
        x1a, (byte)0xea, (byte)0xdd,
    (byte)0xfa, (byte)0xc4, (byte)0x8d, (byte)0x31, (byte)0x2e, (byte)0
        x52, (byte)0xe1, (byte)0x7e,
    (byte)0xd8, (byte)0x8a, (byte)0x53, (byte)0x12, (byte)0xd5, (byte)0
        xac, (byte)0x29, (byte)0x8d,
    (byte)0x57, (byte)0x02, (byte)0xeb, (byte)0xfb, (byte)0xbd, (byte)0
        x06, (byte)0x6a, (byte)0x30,
    (byte)0x73, (byte)0x51, (byte)0x8f, (byte)0x06, (byte)0xb6, (byte)0
        x26, (byte)0x6c, (byte)0xab,
    (byte)0xb1, (byte)0x25, (byte)0x15, (byte)0xa7, (byte)0xb0, (byte)0
        x57, (byte)0x6a, (byte)0x6d,
    (byte)0xbd, (byte)0x66, (byte)0xf2, (byte)0xa2, (byte)0xc7, (byte)0
        x37, (byte)0x24, (byte)0x77,
    (byte)0x4b, (byte)0xbe, (byte)0x9d, (byte)0xf8, (byte)0xb4, (byte)0
        x73, (byte)0xc8, (byte)0x2c,
    (byte)0xc4, (byte)0x5a, (byte)0x64, (byte)0x40, (byte)0x59, (byte)0
        x7d, (byte)0x64, (byte)0x79,
    (byte)0xd4, (byte)0x7f, (byte)0x07, (byte)0x3a, (byte)0x70, (byte)0
        x84, (byte)0xd2, (byte)0xf5,
```

Annexe — ElGamalApplet

```
private static final byte[] X_BYTES = {
    (byte)0x7d, (byte)0xe4, (byte)0xa6, (byte)0x83, (byte)0x3f, (byte)0
    xdf, (byte)0x9e, (byte)0x8e,
    (byte)0x81, (byte)0xb0, (byte)0x89, (byte)0x40, (byte)0x2f, (byte)0
    xad, (byte)0x37, (byte)0x92,
    (byte)0xa3, (byte)0xf0, (byte)0x80, (byte)0xb1, (byte)0x37, (byte)0
    x98, (byte)0x25, (byte)0xc6,
    (byte)0x4e, (byte)0x5a, (byte)0xb7, (byte)0x8c, (byte)0x51, (byte)0
    x58, (byte)0x61, (byte)0x01,
    (byte)0xf7, (byte)0x15, (byte)0x16, (byte)0xa4, (byte)0x4e, (byte)0
    x63, (byte)0x1f, (byte)0x8a,
    (byte)0x6d, (byte)0x51, (byte)0x99, (byte)0x08, (byte)0x9d, (byte)0
    x4c, (byte)0xd3, (byte)0xd8,
    (byte)0x7b, (byte)0x11, (byte)0x62, (byte)0x42, (byte)0x7c, (byte)0
    xe0, (byte)0x97, (byte)0x54,
    (byte)0x55, (byte)0xa8, (byte)0xfb, (byte)0x87, (byte)0x49, (byte)0
    x49, (byte)0xfc, (byte)0x3d,
    (byte)0x31, (byte)0x1e, (byte)0xa9, (byte)0xa0, (byte)0x27, (byte)0
    xa3, (byte)0xe8, (byte)0xc8,
    (byte)0x1f, (byte)0xa2, (byte)0xd3, (byte)0x80, (byte)0x12, (byte)0
    x6f, (byte)0xc0, (byte)0xdc,
    (byte)0xe0, (byte)0x50, (byte)0x23, (byte)0x6f, (byte)0x4f, (byte)0
    x2e, (byte)0x03, (byte)0x8f,
    (byte)0x5e, (byte)0x45, (byte)0x78, (byte)0x2f, (byte)0x8d, (byte)0
    x3f, (byte)0x8f, (byte)0xb2,
    (byte)0xba, (byte)0x22, (byte)0x4e, (byte)0xff, (byte)0x52, (byte)0
    x98, (byte)0x8e, (byte)0xde,
    (byte)0x91, (byte)0x93, (byte)0x36, (byte)0x29, (byte)0x68, (byte)0
    x55, (byte)0xe5, (byte)0x8e,
    (byte)0xc4, (byte)0xc7, (byte)0x3e, (byte)0xca, (byte)0xd5, (byte)0
```

Annexe — ElGamalApplet

```
private static final byte[] Y_BYTES = {
    (byte)0x98, (byte)0x1b, (byte)0xcb, (byte)0xa9, (byte)0xef, (byte)0
    x93, (byte)0x0a, (byte)0xe6,
    (byte)0xbb, (byte)0x3d, (byte)0x52, (byte)0x89, (byte)0xba, (byte)0
    x81, (byte)0xea, (byte)0xa5,
    (byte)0x4d, (byte)0x61, (byte)0xdd, (byte)0x2f, (byte)0x8b, (byte)0
    x77, (byte)0x22, (byte)0x1a,
    (byte)0x32, (byte)0x42, (byte)0xe8, (byte)0x84, (byte)0xf5, (byte)0
    xc1, (byte)0x9f, (byte)0x35,
    (byte)0x1a, (byte)0x10, (byte)0xa9, (byte)0xc3, (byte)0x38, (byte)0
    x91, (byte)0x2d, (byte)0x31,
    (byte)0x49, (byte)0xa3, (byte)0xd1, (byte)0xfc, (byte)0xe8, (byte)0
    x95, (byte)0xfa, (byte)0xb7,
    (byte)0x2d, (byte)0xf1, (byte)0x08, (byte)0x92, (byte)0xe7, (byte)0
    x8d, (byte)0x99, (byte)0xea,
    (byte)0xbc, (byte)0x51, (byte)0xf4, (byte)0xf9, (byte)0x5f, (byte)0
    x81, (byte)0x40, (byte)0x76,
    (byte)0x07, (byte)0xa7, (byte)0x89, (byte)0x61, (byte)0x65, (byte)0
    x74, (byte)0x92, (byte)0x02,
    (byte)0x3b, (byte)0x9b, (byte)0x9d, (byte)0xd4, (byte)0x1f, (byte)0
    xbf, (byte)0x30, (byte)0xba,
    (byte)0xba, (byte)0x46, (byte)0x1c, (byte)0xd7, (byte)0x2f, (byte)0
    xa5, (byte)0xfd, (byte)0xbd,
    (byte)0xaf, (byte)0x81, (byte)0x21, (byte)0xb2, (byte)0xeb, (byte)0
    xca, (byte)0xa9, (byte)0x7e,
    (byte)0xb9, (byte)0x21, (byte)0x19, (byte)0x24, (byte)0x44, (byte)0
    xf5, (byte)0xd8, (byte)0x2c,
    (byte)0x9d, (byte)0xfd, (byte)0x3a, (byte)0xc3, (byte)0xc5, (byte)0
    xea, (byte)0x45, (byte)0xf4,
    (byte)0xd2, (byte)0xe8, (byte)0x74, (byte)0x9f, (byte)0x57, (byte)0
```

```
private final RSAPublicKey rsaKey1;
private final RSAPublicKey rsaKey2;
private final Cipher      rsaCipher;

private final byte[] tmp1;
private final byte[] tmp2;
private final byte[] result256;

private final RandomData rng;

private static void fillZeros(byte[] buffer, short length) {
    for (short i = 0; i < length; i++) {
        buffer[i] = (byte)0x00;
    }
}

public static void install(byte[] bArray, short bOffset, byte bLength) {
    new ElGamalApplet();
}
```

```
protected ElGamalApplet() {
    register();

    rsaKey1 = (RSAPublicKey) KeyBuilder.buildKey(
        KeyBuilder.TYPE_RSA_PUBLIC, KeyBuilder.LENGTH_RSA_1024, false);
    rsaKey2 = (RSAPublicKey) KeyBuilder.buildKey(
        KeyBuilder.TYPE_RSA_PUBLIC, KeyBuilder.LENGTH_RSA_1024, false);

    rsaCipher = Cipher.getInstance(Cipher.ALG_RSA_NOPAD, false);

    tmp1      = JCSysmtem.makeTransientByteArray(KEY_LEN, JCSysmtem.
        CLEAR_ON_RESET);
    tmp2      = JCSysmtem.makeTransientByteArray(KEY_LEN, JCSysmtem.
        CLEAR_ON_RESET);
    result256 = JCSysmtem.makeTransientByteArray((short)(KEY_LEN * 2),
        JCSysmtem.CLEAR_ON_RESET);

    rng = RandomData.getInstance(RandomData.ALG_TRNG);

    rsaKey1.setModulus(P_BYTES, (short)0, KEY_LEN);
    rsaKey2.setModulus(P_BYTES, (short)0, KEY_LEN);
}
```

```
@Override
public Shareable getShareableInterfaceObject(AID clientAID, byte parameter)
{
    if (clientAID == null) return null;
    if (parameter == (byte)0x01 && !isInterfaceShared) {
        isInterfaceShared = true;
        return this;
    }
    return null;
}

@Override
public void process(APDU apdu) {
    if (selectingApplet()) return;
    ISOException.throwIt(ISO7816.SW_COMMAND_NOT_ALLOWED);
}
```


Annexe — ElGamalApplet

```
@Override
public byte[] helloWorld() { return HELLO_WORLD_MESSAGE_2; }

@Override
public byte[] cmdEncrypt(byte[] apduBuff, short len) {
    fillZeros(result256, (short)256);

    generateEphemeralKey();

    rsaKey1.setExponent(kBuffer, (short)0, KEY_LEN);
    rsaCipher.init(rsaKey1, Cipher.MODE_ENCRYPT);
    rsaCipher.doFinal(G_BYTES, (short)1, KEY_LEN, result256, (short)0);

    rsaKey2.setExponent(kBuffer, (short)0, KEY_LEN);
    rsaCipher.init(rsaKey2, Cipher.MODE_ENCRYPT);
    rsaCipher.doFinal(Y_BYTES, (short)0, KEY_LEN, tmp1, (short)0);

    Util.arrayFillNonAtomic(tmp2, (short)0, KEY_LEN, (byte)0);
    Util.arrayCopy(apduBuff, ISO7816.OFFSET_CDATA, tmp2, (short)(KEY_LEN -
        len), len);

    modMulFull(tmp2, tmp1, result256, KEY_LEN);

    return result256;
}
```

```
@Override
public byte[] cmdDecrypt(byte[] apduBuff, short len) {
    fillZeros(result256, (short)256);

    rsaKey1.setExponent(X_BYTES, (short)0, KEY_LEN);
    rsaCipher.init(rsaKey1, Cipher.MODE_ENCRYPT);
    rsaCipher.doFinal(apduBuff, (short)0, KEY_LEN, tmp1, (short)0);

    Util.arrayCopy(P_BYTES, (short)0, result256, (short)0, KEY_LEN);
    result256[(short)(KEY_LEN - 1)] -= 2;
    rsaKey2.setExponent(result256, (short)0, KEY_LEN);
    rsaCipher.init(rsaKey2, Cipher.MODE_ENCRYPT);
    rsaCipher.doFinal(tmp1, (short)0, KEY_LEN, tmp2, (short)0);

    Util.arrayCopy(apduBuff, KEY_LEN, tmp1, (short)0, KEY_LEN);
    modMulFull(tmp1, tmp2, result256, KEY_LEN);
    Util.arrayCopy(result256, KEY_LEN, result256, (short)0, KEY_LEN);

    return result256;
}

private final byte[] mulProduct = new byte[(short)(KEY_LEN * 2)];
private final byte[] mulShifted = new byte[(short)(KEY_LEN * 2)];
```

```
private void modMulFull(byte[] a, byte[] b, byte[] result, short keyLen) {
    short keyLen2 = (short)(keyLen * 2);

    Util.arrayFillNonAtomic(mulProduct, (short)0, keyLen2, (byte)0);
    for (short i = (short)(keyLen - 1); i >= 0; i--) {
        short carry = 0;
        for (short j = (short)(keyLen - 1); j >= 0; j--) {
            short prod = (short)((short)(a[i] & 0xFF) * (short)(b[j] & 0xFF)
                );
            short pos = (short)(i + j + 1);
            prod = (short)(prod + (short)(mulProduct[pos] & 0xFF) + carry);
            mulProduct[pos] = (byte)(prod & 0xFF);
            carry = (short)((prod >> 8) & 0xFF);
        }
        mulProduct[i] = (byte)(carry & 0xFF);
    }

    Util.arrayFillNonAtomic(mulShifted, (short)0, keyLen2, (byte)0);
    Util.arrayCopy(P_BYTES, (short)0, mulShifted, keyLen, keyLen);

    short msbP = msbPos(mulProduct, keyLen2);
    short msbS = msbPos(mulShifted, keyLen2);
    short shifts = (short)(msbS - msbP);
```

```
if (shifts < 0) {
    Util.arrayCopy(mulProduct, keyLen, result, KEY_LEN, keyLen);
    return;
}
if (shifts > (short)(keyLen * 8)) shifts = (short)(keyLen * 8);

for (short i = 0; i < shifts; i++) {
    shiftLeft1(mulShifted, keyLen2);
}

for (short s = (short)(shifts + 1); s > 0; s--) {
    if (compareN(mulProduct, mulShifted, keyLen2) >= 0) {
        subtractN(mulProduct, mulShifted, keyLen2);
    }
    shiftRight1(mulShifted, keyLen2);
}

Util.arrayCopy(mulProduct, keyLen, result, KEY_LEN, keyLen);
}

private final byte[] kBuffer = new byte[KEY_LEN];
```

Annexe — ElGamalApplet

```
private void generateEphemeralKey() {
    for (short attempt = 0; attempt < 32; attempt++) {
        rng.nextBytes(kBuffer, (short)0, KEY_LEN);
        kBuffer[0] = (byte)(kBuffer[0] & (byte)0x7F);
        if (!isZeroOrOne(kBuffer)) return;
    }
    Util.arrayCopy(X_BYTES, (short)0, kBuffer, (short)0, KEY_LEN);
    kBuffer[(short)(KEY_LEN - 1)] ^= (byte)0x55;
}

private void shiftLeft1(byte[] buf, short len) {
    byte carry = 0;
    for (short i = (short)(len - 1); i >= 0; i--) {
        byte newCarry = (byte)((buf[i] & 0xFF) >> 7);
        buf[i] = (byte)(((buf[i] & 0xFF) << 1) | carry);
        carry = newCarry;
    }
}

private void shiftRight1(byte[] buf, short len) {
    byte carry = 0;
    for (short i = 0; i < len; i++) {
        byte newCarry = (byte)(buf[i] & 0x01);
        buf[i] = (byte)(((buf[i] & 0xFF) >>> 1) | (carry << 7));
        carry = newCarry;
    }
}
```

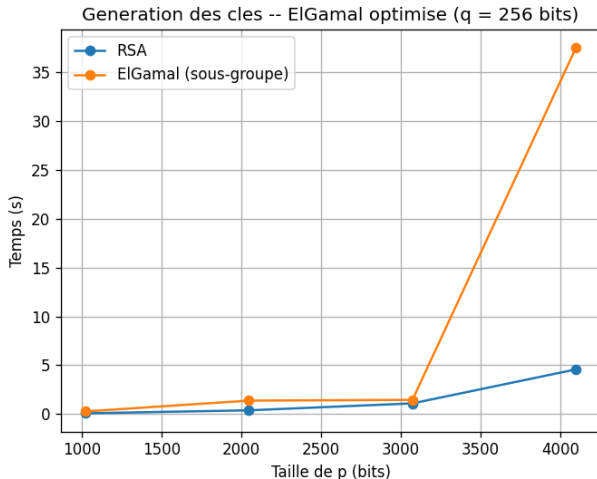
```
private void subtractN(byte[] a, byte[] b, short len) {
    short borrow = 0;
    for (short i = (short)(len - 1); i >= 0; i--) {
        short diff = (short)((short)(a[i] & 0xFF) - (short)(b[i] & 0xFF) -
                               borrow);
        if (diff < 0) { diff += 256; borrow = 1; } else { borrow = 0; }
        a[i] = (byte)(diff & 0xFF);
    }
}

private short compareN(byte[] a, byte[] b, short len) {
    for (short i = 0; i < len; i++) {
        short av = (short)(a[i] & 0xFF);
        short bv = (short)(b[i] & 0xFF);
        if (av < bv) return (short)-1;
        if (av > bv) return (short) 1;
    }
    return 0;
}
```

```
private short msbPos(byte[] buf, short len) {
    for (short i = 0; i < len; i++) {
        if (buf[i] != 0) {
            byte b = buf[i];
            for (short bit = 7; bit >= 0; bit--) {
                if ((b & (byte)(1 << bit)) != 0) {
                    return (short)((short)(i * 8) + (short)(7 - bit));
                }
            }
        }
    }
    return (short)(len * 8);
}

private boolean isZeroOrOne(byte[] a) {
    for (short i = 0; i < (short)(KEY_LEN - 1); i++) {
        if (a[i] != 0) return false;
    }
    return (a[(short)(KEY_LEN - 1)] == 0 || a[(short)(KEY_LEN - 1)] == 1);
}
```

Annexe — Génération de clés (référence)



Génération ElGamal coûteuse à grande taille : recherche de $p = hq + 1$ premier.

Coût *ponctuel*.